

獨自升級Kafka

2025.11.27

Ming-Yen Chung

小明第一天上班拿到了
升級Kafka cluster的任務
他等不及的想好好探索這個卡夫卡

請問小明要怎麼知道現在的Kafka是哪一版呢？

聰明的小明當然知道去 bin 目錄底下找工具用，當然是挑名稱有 version 的來用

```
~/code/kafka_2.13-4.0.1
```

```
> ls bin
```

```
connect-distributed.sh      kafka-delegation-tokens.sh  kafka-replica-verification.sh
connect-mirror-maker.sh    kafka-delete-records.sh     kafka-run-class.sh
connect-plugin-path.sh      kafka-dump-log.sh           kafka-server-start.sh
connect-standalone.sh      kafka-e2e-latency.sh        kafka-server-stop.sh
kafka-acls.sh               kafka-features.sh           kafka-share-groups.sh
kafka-broker-api-versions.sh kafka-get-offsets.sh         kafka-storage.sh
kafka-client-metrics.sh    kafka-groups.sh             kafka-streams-application-reset.sh
kafka-cluster.sh           kafka-jmx.sh                kafka-topics.sh
kafka-configs.sh           kafka-leader-election.sh    kafka-transactions.sh
kafka-console-consumer.sh  kafka-log-dirs.sh           kafka-verifiable-consumer.sh
kafka-console-producer.sh  kafka-metadata-quorum.sh    kafka-verifiable-producer.sh
kafka-console-share-consumer.sh kafka-metadata-shell.sh     trogdor.sh
kafka-consumer-groups.sh   kafka-producer-perf-test.sh windows
kafka-consumer-perf-test.sh kafka-reassign-partitions.sh
```

再問Claude Opus 4.5確認一下

MC

要怎麼知道 當前kafka的版本

查看 Kafka 版本的方法

方法1：直接問 Kafka（最簡單）

```
bash

./bin/kafka-server-start.sh --version

# 或

./bin/kafka-broker-api-versions.sh --version

# 或

./bin/kafka-topics.sh --version
```

```
› ./bin/kafka-broker-api-versions.sh --bootstrap-server localhost:9092 --version
```

4.0.1

```
› ./bin/kafka-topics.sh --version
```

4.0.1

小明：看來還沒有到最新版 4.1 呢，我要來升級一下，好興奮啊

於是小明就被開除了
每天跟老婆還有兒子過著啃米餅的生活



方法 2 : kafka-metadata.sh (KRaft 専用)

```
bash
```

```
./bin/kafka-metadata.sh --bootstrap-server localhost:9092 --command "status"
```

```
# 或
```

```
./bin/kafka-metadata.sh --bootstrap-server localhost:9092 --command "features"
```

```
~/code/kafka_2.13-4.0.1
```

```
> ./bin/kafka-metadata.sh --bootstrap-server localhost:9092 --command "status"  
zsh: no such file or directory: ./bin/kafka-metadata.sh
```

```
~/code/kafka_2.13-4.0.1
```

```
> ./bin/kafka-metadata.sh --bootstrap-server localhost:9092 --command "features"  
zsh: no such file or directory: ./bin/kafka-metadata.sh
```

```
> ./bin/kafka-metadata-shell.sh --snapshot /tmp/kraft-combined-logs/__cluster_metadata-0/00000000000000000000.log --command "cat"
```

```
usage: kafka-metadata-shell [-h] [--snapshot SNAPSHOT] [command [command ...]]
```

```
kafka-metadata-shell: error: unrecognized arguments: '--command'
```


方法 3 : kafka-features.sh

```
bash
```

```
./bin/kafka-features.sh --bootstrap-server localhost:9092 describe
```

```
# 輸出 :
```

```
# Feature: metadata.version SupportedMinVersion: 1 SupportedMaxVersion: 17 FinalizedVersionLevel: 1
```

```
#
```

```
#
```

17 = 3.9-IV0, 所以是 Kafka 3.9

```
~/code/kafka_2.13-4.0.1
```

16:08:56

```
> ./bin/kafka-features.sh --bootstrap-server localhost:9092 describe
```

Feature: eligible.leader.replicas.version	SupportedMinVersion: 0	SupportedMaxVersion: 1	FinalizedVersionLevel: 1	Epoch: 4760
Feature: group.version	SupportedMinVersion: 0	SupportedMaxVersion: 1	FinalizedVersionLevel: 1	Epoch: 4760
Feature: kraft.version	SupportedMinVersion: 0	SupportedMaxVersion: 1	FinalizedVersionLevel: 1	Epoch: 4760
Feature: metadata.version	SupportedMinVersion: 3.3-IV3	SupportedMaxVersion: UNKNOWN 29	FinalizedVersionLevel: UNKNOWN 29	Epoch: 4760
Feature: share.version	SupportedMinVersion: 0	SupportedMaxVersion: 1	FinalizedVersionLevel: 1	Epoch: 4760
Feature: streams.version	SupportedMinVersion: 0	SupportedMaxVersion: 1	FinalizedVersionLevel: 1	Epoch: 4760
Feature: transaction.version	SupportedMinVersion: 0	SupportedMaxVersion: 2	FinalizedVersionLevel: 2	Epoch: 4760

UNKNOWN 29是什麼
IV3又是什麼

UNKNOWN: 舊的Tool看不懂啦

29 跟 IV3 請查看MetadataVersion的code

MetadataVersion

/**

- * This class contains the different Kafka versions.
- * Right now, we use them for upgrades - users can configure the version of the API brokers will use to communicate between themselves.
- * This is only for inter-broker communications - when communicating with clients, the client decides on the API version.
- *

- * Note that the ID we initialize for each version is important.
- * We consider a version newer than another if it is lower in the enum list (to avoid depending on lexicographic order)
- *

- * Since the api protocol may change more than once within the same release and to facilitate people deploying code from trunk, we have the concept of internal versions (first introduced during the 1.0 development cycle). For example,
- * the first time we introduce a version change in a release, say 1.0, we will add a config value "1.0-IV0" and a corresponding enum constant IBP_1_0-IV0. We will also add a config value "1.0" that will be mapped to the latest internal version object, which is IBP_1_0-IV0. When we change the protocol a second time while developing 1.0, we will add a new config value "1.0-IV1" and a corresponding enum constant IBP_1_0-IV1. We will change the config value "1.0" to map to the latest internal version IBP_1_0-IV1. The config value of "1.0-IV0" is still mapped to IBP_1_0-IV0. This way, if people are deploying from trunk, they can use "1.0-IV0" and "1.0-IV1" to upgrade one internal version at a time. For most people who just want to use released version, they can use "1.0" when upgrading to the 1.0 release.

*/

MetadataVersion

MetadataVersion 摘要

這段 JavaDoc 在說什麼？

MetadataVersion 的用途

1. 控制 Broker 之間的通訊協議版本
 - └ 注意：Client ↔ Broker 的版本由 Client 決定
2. 用於升級
 - └ 讓不同版本的 Broker 能共存

MetadataVersion

Internal Version (IV) 的概念

為什麼需要 IV ?

問題：同一個 Release 週期內，協議可能改很多次

例如開發 Kafka 3.9 時：

第一次改協議 → 3.9-IV0

第二次改協議 → 3.9-IV1

第三次改協議 → 3.9-IV2

最後 Release 時：

"3.9" 這個 config 值 → 自動對應到最新的 3.9-IV2

MetadataVersion

版本對應關係

Config 值		Enum 常數		Feature Level
"3.9"	→	IBP_3_9_IV0	→	17
"3.9-IV0"	→	IBP_3_9_IV0	→	17
"4.0"	→	IBP_4_0_IV3	→	21 (假設最新是 IV3)
"4.0-IV0"	→	IBP_4_0_IV0	→	18
"4.0-IV1"	→	IBP_4_0_IV1	→	19
"4.0-IV2"	→	IBP_4_0_IV2	→	20
"4.0-IV3"	→	IBP_4_0_IV3	→	21

開發中的 trunk :

"4.1-IV0"	→	IBP_4_1_IV0	→	27 (你遇到的版本)
-----------	---	-------------	---	-------------

所以我用的binary太舊了

改成4.1就看的清楚了

```
> ./bin/kafka-features.sh --bootstrap-server localhost:9092 describe
Feature: eligible.leader.replicas.version      SupportedMinVersion: 0   SupportedMaxVersion: 1   FinalizedVersionLevel: 1   Epoch: 55
Feature: group.version      SupportedMinVersion: 0   SupportedMaxVersion: 1   FinalizedVersionLevel: 1   Epoch: 55
Feature: kraft.version      SupportedMinVersion: 0   SupportedMaxVersion: 1   FinalizedVersionLevel: 1   Epoch: 55
Feature: metadata.version   SupportedMinVersion: 3.3-IV3   SupportedMaxVersion: 4.1-IV1   FinalizedVersionLevel: 4.1-IV1   Epoch: 55
Feature: share.version      SupportedMinVersion: 0   SupportedMaxVersion: 1   FinalizedVersionLevel: 0   Epoch: 55
Feature: transaction.version SupportedMinVersion: 0   SupportedMaxVersion: 2   FinalizedVersionLevel: 2   Epoch: 55
```

Upgrading ZooKeeper-based clusters (3.9)

- ~~1. Update server properties on all brokers and add the following properties. CURRENT_KAFKA_VERSION refers to the version you are upgrading from. CURRENT_MESSAGE_FORMAT_VERSION refers to the message format version currently in use. If you have previously overridden the message format version, you should keep its current value. Alternatively, if you are upgrading from a version prior to 0.11.0.x, then CURRENT_MESSAGE_FORMAT_VERSION should be set to match CURRENT_KAFKA_VERSION.~~
 - ~~inter.broker.protocol.version=CURRENT_KAFKA_VERSION (e.g. 3.8, 3.7, etc.)~~
 - ~~log.message.format.version=CURRENT_MESSAGE_FORMAT_VERSION (See [potential performance impact following the upgrade](#) for the details on what this configuration does.)~~
- If you are upgrading from version 2.4.0** or above, and you have not overridden the message format, then you only need to override the inter-broker protocol version. However, before doing this, make sure ZooKeeper is upgraded to version 3.8.3 or higher.
 - inter.broker.protocol.version=CURRENT_KAFKA_VERSION (e.g. 3.8, 3.7, etc.)
- Upgrade the brokers one at a time: shut down the broker, update the code, and **restart** it. Once you have done so, the brokers will be running the latest version and you can verify that the cluster's behavior and performance meets expectations. It is still possible to downgrade at this point if there are any problems.
- Once the cluster's behavior and performance has been verified, bump the protocol version by editing `inter.broker.protocol.version` and setting it to **3.9**.
- Restart** the brokers one by one for the new protocol version to take effect. Once the brokers begin using the latest protocol version, it will no longer be possible to downgrade the cluster to an older version.
- ~~If you have overridden the message format version as instructed above, then you need to do one more rolling restart to upgrade it to its latest version. Once all (or most) consumers have been upgraded to 0.11.0 or later, change log.message.format.version to 3.8 on each broker and restart them one by one. Note that the older Scala clients, which are no longer maintained, do not support the message format introduced in 0.11, so to avoid conversion costs (or to take advantage of [exactly once semantics](#)), the newer Java clients must be used.~~

Upgrading KRaft-based clusters (3.9)

1. Upgrade the brokers one at a time: shut down the broker, update the code, and **restart** it. Once you have done so, the brokers will be running the latest version and you can verify that the cluster's behavior and performance meets expectations.
2. Once the cluster's behavior and performance has been verified, bump the metadata.version by running
`bin/kafka-features.sh upgrade --metadata 3.9`

Demo 3.8 -> 3.9

Start a 3.8 Kraft Kafka

```
export KAFKA_REPO=~/.code/kafka_2.13-3.8.1
cd $KAFKA_REPO
rm -rf /tmp/kraft-combined-logs
KAFKA_CLUSTER_ID="$$(./bin/kafka-storage.sh random-uuid)"
bin/kafka-storage.sh format -t $KAFKA_CLUSTER_ID -c config/kraft/server.properties
bin/kafka-server-start.sh config/kraft/server.properties
```

Create and write events into the topic

```
bin/kafka-topics.sh --create --topic test --bootstrap-server localhost:9092
bin/kafka-console-producer.sh --topic test --bootstrap-server localhost:9092
```

Consume the events

```
bin/kafka-console-consumer.sh --topic test --from-beginning --bootstrap-server localhost:9092
```

Check the version

```
./bin/kafka-features.sh --bootstrap-server localhost:9092 describe
```

Demo

Upgrade the brokers one at a time

1. shut down the 3.8.1 broker

2. update the code

```
cd kafka_2.13-3.9.1
```

3. and restart it

```
bin/kafka-server-start.sh config/kraft/server.properties
```

4. bump the metadata.version

```
bin/kafka-features.sh --bootstrap-server localhost:9092 upgrade --metadata 3.9
```

如果大禮包有你不想要的禮物怎麼辦

KIP-1022: Formatting and Updating Features

- 4.0開始採用
- 很多功能都可以擁有自己獨立的feature, 不用跟metadataVersion綁一起了
 - e.g,
 - New group coordinator: **group.version**
 - share group: **share.version**
- 想要全部一起升級, 採用合適的default value: 從 `--metadata` 改成 `--release-version`
- 可以查看dependencies

3.9

```
> ./bin/kafka-features.sh --bootstrap-server localhost:9092 describe
Feature: kraft.version SupportedMinVersion: 0 SupportedMaxVersion: 1 FinalizedVersionLevel: 1 Epoch: 29
Feature: metadata.version SupportedMinVersion: UNKNOWN 1 SupportedMaxVersion: 3.9-IV0 FinalizedVersionLevel: 3.9-IV0 Epoch: 29
```

4.0

```
> ./bin/kafka-features.sh --bootstrap-server localhost:9092 describe
Feature: eligible.leader.replicas.version SupportedMinVersion: 0 SupportedMaxVersion: 1 FinalizedVersionLevel: 0 Epoch: 16
Feature: group.version SupportedMinVersion: 0 SupportedMaxVersion: 1 FinalizedVersionLevel: 1 Epoch: 16
Feature: kraft.version SupportedMinVersion: 0 SupportedMaxVersion: 1 FinalizedVersionLevel: 1 Epoch: 16
Feature: metadata.version SupportedMinVersion: 3.3-IV3 SupportedMaxVersion: 4.0-IV3 FinalizedVersionLevel: 4.0-IV3 Epoch: 16
Feature: transaction.version SupportedMinVersion: 0 SupportedMaxVersion: 2 FinalizedVersionLevel: 2 Epoch: 16
```

4.1

```
> ./bin/kafka-features.sh --bootstrap-server localhost:9092 describe
Feature: eligible.leader.replicas.version SupportedMinVersion: 0 SupportedMaxVersion: 1 FinalizedVersionLevel: 1 Epoch: 17
Feature: group.version SupportedMinVersion: 0 SupportedMaxVersion: 1 FinalizedVersionLevel: 1 Epoch: 17
Feature: kraft.version SupportedMinVersion: 0 SupportedMaxVersion: 1 FinalizedVersionLevel: 1 Epoch: 17
Feature: metadata.version SupportedMinVersion: 3.3-IV3 SupportedMaxVersion: 4.1-IV1 FinalizedVersionLevel: 4.1-IV1 Epoch: 17
Feature: share.version SupportedMinVersion: 0 SupportedMaxVersion: 1 FinalizedVersionLevel: 0 Epoch: 17
Feature: transaction.version SupportedMinVersion: 0 SupportedMaxVersion: 2 FinalizedVersionLevel: 2 Epoch: 17
```

4.2

```
> ./bin/kafka-features.sh --bootstrap-server localhost:9092 describe
Feature: eligible.leader.replicas.version SupportedMinVersion: 0 SupportedMaxVersion: 1 FinalizedVersionLevel: 1 Epoch: 17
Feature: group.version SupportedMinVersion: 0 SupportedMaxVersion: 1 FinalizedVersionLevel: 1 Epoch: 17
Feature: kraft.version SupportedMinVersion: 0 SupportedMaxVersion: 1 FinalizedVersionLevel: 1 Epoch: 17
Feature: metadata.version SupportedMinVersion: 3.3-IV3 SupportedMaxVersion: 4.2-IV1 FinalizedVersionLevel: 4.2-IV1 Epoch: 17
Feature: share.version SupportedMinVersion: 0 SupportedMaxVersion: 1 FinalizedVersionLevel: 1 Epoch: 17
Feature: streams.version SupportedMinVersion: 0 SupportedMaxVersion: 1 FinalizedVersionLevel: 1 Epoch: 17
Feature: transaction.version SupportedMinVersion: 0 SupportedMaxVersion: 2 FinalizedVersionLevel: 2 Epoch: 17
```

~/code/kafka trunk *2 ?1

```
> ./bin/kafka-features.sh --bootstrap-server localhost:9092 version-mapping
metadata.version=29 (4.2-IV1)
kraft.version=1
transaction.version=2
group.version=1
eligible.leader.replicas.version=1
share.version=1
streams.version=1
```

~/code/kafka trunk *2 ?1

```
> ./bin/kafka-features.sh --bootstrap-server localhost:9092 version-mapping --release-version 4.1
metadata.version=27 (4.1)
kraft.version=1
transaction.version=2
group.version=1
eligible.leader.replicas.version=1
share.version=0
streams.version=0
```

~/code/kafka trunk *2 ?1

```
> ./bin/kafka-features.sh --bootstrap-server localhost:9092 version-mapping --release-version 4.0
metadata.version=25 (4.0)
kraft.version=1
transaction.version=2
group.version=1
eligible.leader.replicas.version=0
share.version=0
streams.version=0
```

假設我要從4.1降到4.0, 用4.0的binary來downgrade, 可以嗎？

```
› cd kafka_2.13-4.0.1
```

```
› ./bin/kafka-features.sh --bootstrap-server localhost:9092 downgrade --release-version 4.0
```


如果用舊的binary執行downgrade

```
~/code/kafka_2.13-4.0.1 17:22:29
> ./bin/kafka-features.sh --bootstrap-server localhost:9092 describe
Feature: eligible.leader.replicas.version      SupportedMinVersion: 0   SupportedMaxVersion: 1   FinalizedVersionLevel: 1   Epoch: 13535
Feature: group.version      SupportedMinVersion: 0   SupportedMaxVersion: 1   FinalizedVersionLevel: 1   Epoch: 13535
Feature: kraft.version      SupportedMinVersion: 0   SupportedMaxVersion: 1   FinalizedVersionLevel: 1   Epoch: 13535
Feature: metadata.version   SupportedMinVersion: 3.3-IV3   SupportedMaxVersion: UNKNOWN 29   FinalizedVersionLevel: UNKNOWN 29   Epoch: 13535
Feature: share.version      SupportedMinVersion: 0   SupportedMaxVersion: 1   FinalizedVersionLevel: 1   Epoch: 13535
Feature: streams.version    SupportedMinVersion: 0   SupportedMaxVersion: 1   FinalizedVersionLevel: 1   Epoch: 13535
Feature: transaction.version SupportedMinVersion: 0   SupportedMaxVersion: 2   FinalizedVersionLevel: 2   Epoch: 13535
```

```
~/code/kafka_2.13-4.0.1 17:23:07
> ./bin/kafka-features.sh --bootstrap-server localhost:9092 downgrade --release-version 4.0
eligible.leader.replicas.version was downgraded to 0.
group.version was downgraded to 1.
kraft.version was downgraded to 1.
metadata.version was downgraded to 25.
transaction.version was downgraded to 2.
```

```
~/code/kafka_2.13-4.0.1 17:23:14
> ./bin/kafka-features.sh --bootstrap-server localhost:9092 describe
Feature: eligible.leader.replicas.version      SupportedMinVersion: 0   SupportedMaxVersion: 1   FinalizedVersionLevel: 0   Epoch: 13558
Feature: group.version      SupportedMinVersion: 0   SupportedMaxVersion: 1   FinalizedVersionLevel: 1   Epoch: 13558
Feature: kraft.version      SupportedMinVersion: 0   SupportedMaxVersion: 1   FinalizedVersionLevel: 1   Epoch: 13558
Feature: metadata.version   SupportedMinVersion: 3.3-IV3   SupportedMaxVersion: UNKNOWN 29   FinalizedVersionLevel: 4.0-IV3   Epoch: 13558
Feature: share.version      SupportedMinVersion: 0   SupportedMaxVersion: 1   FinalizedVersionLevel: 1   Epoch: 13558
Feature: streams.version    SupportedMinVersion: 0   SupportedMaxVersion: 1   FinalizedVersionLevel: 1   Epoch: 13558
Feature: transaction.version SupportedMinVersion: 0   SupportedMaxVersion: 2   FinalizedVersionLevel: 2   Epoch: 13558
```

Upgrading KRaft-based clusters (4.1)

1. Upgrade the brokers one at a time: shut down the broker, update the code, and restart it. Once you have done so, the brokers will be running the latest version and you can verify that the cluster's behavior and performance meet expectations.
2. Once the cluster's behavior and performance have been verified, finalize the upgrade by running
`bin/kafka-features.sh --bootstrap-server localhost:9092 upgrade --release-version 4.0`
3. Note that cluster metadata downgrade is not supported in this version since it has metadata changes. Every [MetadataVersion](#) has a boolean parameter that indicates if there are metadata changes (i.e. `IBP_4_0_IV1(23, "4.0", "IV1", true)` means this version has metadata changes). Given your current and target versions, a downgrade is only possible if there are no metadata changes in the versions between.

Upgrading KRaft-based clusters (3.9)

1. Upgrade the brokers one at a time: shut down the broker, update the code, and restart it. Once you have done so, the brokers will be running the latest version and you can verify that the cluster's behavior and performance meets expectations.
2. Once the cluster's behavior and performance **has** been verified, bump the metadata.version by running
`bin/kafka-features.sh upgrade --metadata 3.9`
3. Note that cluster metadata downgrade is not supported in this software version. Every [MetadataVersion](#) after 3.2.x has a boolean parameter that indicates if there are metadata changes (i.e. `IBP_3_3_IV3(7, "3.3", "IV3", true)` means this version has metadata changes). Given your current and target versions, a downgrade is only possible if there are no metadata changes in the versions between.

Upgrading KRaft-based clusters (4.1)

1. Upgrade the brokers one at a time: shut down the broker, update the code, and restart it. Once you have done so, the brokers will be running the latest version and you can verify that the cluster's behavior and performance meet expectations.
2. Once the cluster's behavior and performance **have** been verified, finalize the upgrade by running
`bin/kafka-features.sh --bootstrap-server localhost:9092 upgrade --release-version 4.0`
3. Note that cluster metadata downgrade is not supported in this version since it has metadata changes. Every [MetadataVersion](#) has a boolean parameter that indicates if there are metadata changes (i.e. `IBP_4_0_IV1(23, "4.0", "IV1", true)` means this version has metadata changes). Given your current and target versions, a downgrade is only possible if there are no metadata changes in the versions between.

小明覺得Kafka 4.0太可怕了，想降回3.9請問他辦得到嗎

```
./bin/kafka-features.sh --bootstrap-server localhost:9092 downgrade --release-version 3.9
```

小明覺得Kafka 4.1太可怕了，想降回4.0請問他辦得到嗎

```
./bin/kafka-features.sh --bootstrap-server localhost:9092 downgrade --release-version 4.0
```

Note that cluster metadata downgrade is not supported in this software version. Every [MetadataVersion](#) after 3.2.x has a boolean parameter that indicates **if there are metadata changes** (i.e. `IBP_3_3_IV3(7, "3.3", "IV3", true)` means this version has metadata changes). Given your current and target versions, a downgrade is only possible if there are no metadata changes in the versions between.

Downgrade to older (changed) metadata

我想應該是沒機會了

KIP-1155: Metadata Version Downgrades

Created by Colin McCabe, last modified by Jonah Hooper on Sep 22, 2025

Status

Current state: Under discussion

Vote thread:

Discussion thread: <https://lists.apache.org/thread/cwhkf26np3kjt33yhlsvp3k3jf7ofgf3>

JIRA:  [KAFKA-19104](#) - Support metadata version downgrade in Kafka [OPEN](#)

<https://cwiki.apache.org/confluence/display/KAFKA/KIP-1155%3A+Metadata+Version+Downgrades>

Takeaway

- 怎麼查看broker版本
- Kraft升級少了一次改inter.broker.protocol.version config的重啟
- MetadataVersion以及他對應的IV是什麼, 怎樣的情況可以降級
- 4.0新導入以後之後出現的各種feature是什麼東西

Reference

- <https://medium.com/google-cloud/what-version-of-apache-kafka-are-you-running-is-a-hard-question-153976b14030>

